

Project Plan

Coordinating Speculative Agent Workloads
Over Data Backends

Research Focus:

**Middleware coordination policies for
redundancy elimination in multi-agent
database query workloads**

Prepared: March 23, 2026

Duration: 4 months (16 weeks)

Benchmark: BIRD Text-to-SQL Dataset

1. RESEARCH QUESTION

What is the most effective architecture for coordinating speculative agent workloads over data backends, and where are the trade-offs between coordination overhead and redundancy elimination?

This project designs and evaluates a middleware layer that sits between a pool of LLM-based agents and one or more database backends. By implementing a spectrum of coordination policies — from no sharing to phase-aware broadcasting — and measuring their impact on query volume, latency, accuracy, and token cost, the thesis will characterise the trade-off space that determines when coordination helps and when it hurts.

2. SYSTEM ARCHITECTURE

The middleware comprises four components, each with a well-defined role.

Query Interceptor. Captures every SQL query before it reaches the database. It parses the statement, produces a normalised representation (canonicalising whitespace, alias names, and predicate ordering), and checks the cache. On a hit the cached result is returned directly; on a miss the query is forwarded, the result stored, and made available to other agents. **Correction:** the original plan underestimated the difficulty of SQL normalisation — aliasing and predicate reordering require an AST-level comparison, not simple string canonicalisation. The implementation should use *sqlglot* to parse queries into an AST before hashing.

Shared Discovery Board. A lightweight key–value store where agents post high-level findings — which tables are relevant, what column naming conventions the schema uses, which exploratory approaches failed. This operates at a semantic level above the query cache: an agent that discovers sales data lives in `rev_quarterly` rather than `sales` posts that mapping so others skip the search.

Coordination Policy Engine. Decides how aggressively to share information among agents. The set of policies tested here constitutes the core research contribution. Each policy produces a different position on the trade-off curve between coordination overhead and redundancy savings.

Monitoring & Metrics Layer. Logs every query issued, cache hit/miss, board read/write, per-stage latency, and agent accuracy. All evaluation results derive from this instrumentation.

3. COORDINATION POLICIES UNDER TEST

P0 — No Coordination (Baseline). Agents work independently. Every query hits the database with no information sharing. This reproduces the default multi-agent setting and establishes the redundancy baseline.

P1 — Passive Caching. The middleware caches query results keyed on normalised SQL. Agents are unaware of each other but benefit from cache hits on identical queries. This quantifies trivially eliminable redundancy.

P2 — Active Result Sharing. When Agent A receives a result, the middleware pushes it to Agents B and C if they are working on the same task, injecting it into their context as additional grounding. This tests

whether unsolicited information helps or introduces noise. **Correction:** context-window budget must be managed carefully — inject results as concise summaries (schema + row counts + sample rows) rather than full result sets to avoid crowding out the agent's own reasoning.

P3 — Discovery Broadcasting. Agents post and read high-level findings via the shared discovery board. The hypothesis is that semantic-level sharing (e.g. 'the date column uses YYYY-MM-DD format') is more useful than raw query results.

P4 — Staged Coordination. Combines policies by agent phase. During exploration, discoveries are broadcast aggressively (P3). During solution formulation, only passive caching (P1) is active so that agents are not biased toward identical solutions. Phase detection uses a rule-based classifier: queries touching metadata tables or using small LIMIT values indicate exploration; full aggregations and joins indicate solution formulation. **Correction:** the original plan's phase-detection heuristic assumed agents query `information_schema` directly — this should be generalised to also catch PRAGMA queries (DuckDB) and schema-inspection functions used by different LLM tool-calling patterns.

4. MONTH-BY-MONTH TIMELINE

Key correction applied: the original plan allocated only two weeks for thesis writing. This revised plan begins writing in Month 3 and reserves the full final month for writing, analysis, and buffer. Agent counts have been unified to 3, 10, and 25 across all experiments for consistency.

Month 1 — Infrastructure & Baselines

Weeks 1–2: Benchmark infrastructure

- Install DuckDB; load the BIRD text-to-SQL dataset.
- Build an async Python orchestrator (prefer lightweight `asyncio` over LangGraph to minimise framework lock-in).
- Get a single agent solving BIRD tasks end-to-end with full query logging.
- Set up version control and reproducibility tooling (fixed random seeds, pinned library versions, experiment-tracking via Weights & Biases or MLflow).

Weeks 3–4: Multi-agent baselines

- Run 3, 10, and 25 parallel agents on the same tasks.
- Log every query; build analysis scripts measuring total queries, unique queries, sub-expression overlap, and time to solution.
- Reproduce the reference paper's redundancy analysis to confirm similar patterns.
- **Deliverable:** baseline redundancy report (thesis Chapter 2 data).

Month 2 — Middleware Implementation

Weeks 1–2: Query interceptor & passive cache

- Use `sqlglot` to parse SQL into an AST; canonicalise and hash for cache keys.
- Implement exact-match caching; optionally experiment with sub-query matching if time allows (flag as stretch goal).
- Re-run baseline workloads and measure cache elimination rate and speedup.

Weeks 3–4: Discovery board & active sharing

- Implement the shared discovery board as a task-keyed SQLite store.

- Build the injection mechanism: summarise results (schema, row count, sample rows) before appending to agent context.
- Run experiments: passive caching alone vs. passive caching + active sharing.
- **Deliverable:** working middleware with P0–P2 policies.

Month 3 — Advanced Policies, Evaluation & Writing Begins

Weeks 1–2: Discovery broadcasting & staged coordination

- Implement the rule-based phase classifier (metadata / PRAGMA / LIMIT → exploration; full SELECT → formulation). Generalise to cover DuckDB-specific patterns.
- Wire P3 and P4 policies into the middleware.
- Preliminary experiments to validate phase detection accuracy.

Weeks 3–4: Full evaluation matrix & start thesis draft

- Run every policy (P0–P4) × agent count (3, 10, 25) × at least 2 LLMs.
- Metrics per configuration: total queries, cache hit rate, database load, wall-clock time, agent accuracy (execution accuracy on BIRD), token cost, coordination overhead (middleware latency).
- Report 95% confidence intervals (bootstrap) for accuracy comparisons.
- Begin thesis draft: introduction, background, and system design chapters.

Month 4 — Analysis, Writing & Buffer

Weeks 1–2: Targeted follow-ups & analysis

- Identify surprising findings and run targeted experiments (e.g. if active sharing hurts strong models but helps weak ones, investigate why).
- Produce key figures and tables; complete the evaluation chapter.
- Draft discussion chapter: what worked, what didn't, and why.

Weeks 3–4: Finalise thesis & buffer

- Complete conclusion and future-work sections.
- Full revision pass; advisor review cycle.
- Buffer time for unexpected issues or additional experiments.
- **Deliverable:** submitted thesis.

5. TECHNICAL DECISIONS

Component	Choice	Rationale
Primary LLM	GPT-4o-mini	Cost-effective at scale; referenced in prior work. Monitor pricing — set a hard budget cap.
Secondary LLM	Claude 3.5 Haiku or Qwen2.5-Coder-7B	API-based if no GPU; open-weight if local GPU available. Provides model-family diversity.
Query Cache	In-memory dict, AST-normalised keys via sqlglot	Exact matching captures most redundancy; AST normalisation avoids false misses from cosmetic SQL differences.
Discovery Board	SQLite (task-keyed)	Persistent, queryable, minimal overhead. Research question is about policies, not store engineering.

Component	Choice	Rationale
Phase Detector	Rule-based classifier	Metadata/PRAGMA/LIMIT queries → exploration. Full aggregations/joins → formulation. No ML needed.
Orchestrator	Async Python (asyncio)	Lightweight, no framework lock-in. Prefer over LangGraph unless graph-based workflows prove necessary.
Database	DuckDB	In-process, fast analytical queries, no server management. Ideal for reproducible benchmarks.
Experiment Tracking	W&B or MLflow	Log hyperparameters, metrics, and artefacts for reproducibility.

6. EVALUATION METRICS

Each configuration is measured on seven dimensions. All accuracy comparisons must include 95% bootstrap confidence intervals to ensure statistical rigour.

- **Total queries issued** — raw count of SQL statements sent by all agents.
- **Cache hit rate** — fraction of queries served from cache (P1–P4 only).
- **Database load** — queries actually executed against DuckDB after caching.
- **Wall-clock time** — end-to-end latency from task dispatch to final answer.
- **Execution accuracy** — percentage of BIRD tasks answered correctly (standard metric).
- **Token cost** — total input + output tokens across all agents for the task.
- **Coordination overhead** — additional latency introduced by middleware processing.

7. RISK REGISTER & MITIGATIONS

Risk	Likelihood	Impact	Mitigation
API cost overrun	Medium	High	Set hard budget caps per experiment. Reduce configurations; prioritise P0 vs P1 vs P4. Use smaller BIRD subsets for exploratory runs.
API rate limits	High	Medium	Implement exponential back-off with jitter. Run agents in staggered batches if needed. Pre-compute which tasks to run to avoid wasted calls.
Lower-than-expected redundancy	Low	Medium	Report as a finding. Investigate whether agent diversity or task simplicity drives the result. Compare against reference paper's figures.
Active sharing hurts accuracy	Medium	Low	This is a valuable finding — it suggests redundancy provides beneficial diversity. Characterise the threshold and write up as a key contribution.
SQL normalisation edge cases	High	Low	Use sqlglot's AST for normalisation. Accept that some near-miss queries will not cache-hit; log and report the miss rate.
Timeline slippage	Medium	High	Writing starts in Month 3 (not Month 4). Month 4 includes buffer weeks. Scope down to P0/P1/P4 comparison if behind schedule.

8. CORRECTIONS & IMPROVEMENTS APPLIED

The following corrections and improvements have been applied relative to the original project description.

1. **SQL normalisation underestimated.** Simple string canonicalisation will miss semantically identical queries with different alias names or predicate ordering. Corrected to use `sqlglot` AST-level normalisation.
2. **Inconsistent agent counts.** The original plan used 10/20/50 for baselines but 3/10/25 for the evaluation matrix. Unified to 3, 10, and 25 throughout.
3. **Thesis writing timeline unrealistic.** Two weeks is insufficient for a complete thesis. Writing now begins in Month 3 (weeks 3–4) and continues through all of Month 4, with buffer time built in.
4. **No statistical rigour specified.** Added requirement for 95% bootstrap confidence intervals on all accuracy comparisons.
5. **Phase detection too narrow.** The heuristic assumed agents query `information_schema`, but DuckDB agents may use PRAGMA statements or other introspection functions. Generalised the classifier.
6. **Context-window management absent.** Active result sharing (P2) risks filling agent context with raw data. Corrected to inject concise summaries instead of full result sets.
7. **Missing reproducibility practices.** Added experiment tracking (W&B / MLflow), fixed random seeds, and pinned library versions.
8. **No API rate-limit mitigation.** Added exponential back-off with jitter and staggered batch execution to the risk register.
9. **No reference paper citation.** The plan references 'the paper' repeatedly without a formal citation. A clear citation should be included in the thesis introduction.
10. **Framework recommendation softened.** LangGraph may introduce unnecessary complexity. Recommended a lightweight `asyncio` orchestrator as the default, with LangGraph as a fallback only if graph-based workflows prove necessary.

9. THESIS STRUCTURE

Chapter 1 — Introduction & Motivation. Frame the problem: LLM agents querying databases generate massive query redundancy. Motivate the middleware approach. State the research question.

Chapter 2 — Background. Agentic speculation, text-to-SQL benchmarks (BIRD), distributed caching and coordination models, related work.

Chapter 3 — System Design. Architecture of the four middleware components. Formal description of each coordination policy (P0–P4). Implementation choices.

Chapter 4 — Evaluation. Experimental setup, full results matrix, trade-off analysis across all seven metrics. Statistical tests. Key figures and tables.

Chapter 5 — Discussion. What worked, what didn't, and why. Connection to distributed-systems principles (consistency models, congestion control analogies). Limitations.

Chapter 6 — Conclusion & Future Work. Summary of contributions. Directions: semantic cache matching, learned policies, multi-database federation.

10. KEY MILESTONES & DELIVERABLES

Week	Milestone	Deliverable
4	Baselines complete	Redundancy report; baseline figures
8	Middleware P0–P2 operational	Cache elimination and speedup measurements
10	All policies (P0–P4) implemented	Phase-detection validation results
12	Full evaluation matrix complete	Results tables, key figures; draft of Chapters 1–3
14	Analysis & follow-ups done	Evaluation and discussion chapters drafted
16	Thesis submitted	Complete thesis document

This plan is scoped to be achievable within a 4-month timeline with a single researcher. The research contribution is bounded and evaluable: characterising the trade-off space between coordination overhead and redundancy elimination in multi-agent database workloads.